COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE010 DATA STRUCTURES AND ALGORITHMS	**SECTION:**	CPE21S3
DATE PERFORMED:	OCTOBER 4 2026	**DATE SUBMITTED:**	OCTOBER 6 2026
NAME:	DUSTIN CARL B. MAPAS	**INSTRUCTOR:**	ENGR. JIMLORD QUEAJDO

ACTIVITY NO. 10.1 **Hands-on Activity 10.1 Graphs**

1. PROCEDURE OUTPUT

ILO A

Main File

```cpp
#include <iostream>

// stores adjacency list items
struct adjNode {
    int val, cost;
    adjNode* next;
};

// structure to store edges
struct graphEdge {
    int start_ver, end_ver, weight;
};

class DiaGraph {
    // insert new nodes into adjacency list from given graph
    adjNode* getAdjListNode(int value, int weight, adjNode* head) {
        adjNode* newNode = new adjNode;
        newNode->val = value;
        newNode->cost = weight;
        newNode->next = head; // point new node to current head
        return newNode;
    }

    int N; // number of nodes in the graph

public:
    adjNode **head;

    // Constructor
    // adjacency list as array of pointers
    DiaGraph(graphEdge edges[], int n, int N) {
        // allocate new node
        head = new adjNode*[N]();
        this->N = N;

        // initialize head pointer for all vertices
        for (int i = 0; i < N; ++i)
            head[i] = nullptr;
```

```
        // construct directed graph by adding edges to it
        for (unsigned i = 0; i < n; i++) {
            int start_ver = edges[i].start_ver;
            int end_ver = edges[i].end_ver;
            int weight = edges[i].weight;

            // insert in the beginning
            adjNode* newNode = getAdjListNode(value:end_ver, weight, head[start_ver]);

            // point head pointer to new node
            head[start_ver] = newNode;
        }
    }

    // Destructor
    ~DiaGraph() {
        for (int i = 0; i < N; i++)
            delete[] head[i];
        delete[] head;
    }
};

// print all adjacent vertices of given vertex
void display_AdjList(adjNode* ptr, int i) {
    while (ptr != nullptr) {
        std::cout << "(" << i << ", " << ptr->val << ", " << ptr->cost << ") ";
        ptr = ptr->next;
    }
    std::cout << std::endl;
}

```

```cpp
// graph implementation
int main() {
    // graph edges array.
    // (x, y, w) -> edge from x to y with weight w
    graphEdge edges[] = {
        { .start_ver = 0, .end_ver = 1, .weight = 2},
    { .start_ver = 0, .end_ver = 2, .weight = 4},
    { .start_ver = 1, .end_ver = 4, .weight = 3},
    { .start_ver = 2, .end_ver = 3, .weight = 2},
    { .start_ver = 3, .end_ver = 1, .weight = 4},
    { .start_ver = 4, .end_ver = 3, .weight = 3}
    };

    int N = 6; // Number of vertices in the graph
    // calculate number of edges
    int n = sizeof(edges) / sizeof(edges[0]);

    // construct graph
    DiaGraph diagraph(edges, n, N);

    // print adjacency list representation of graph
    std::cout << "Graph adjacency list " << std::endl
              << "(start_vertex, end_vertex, weight): " << std::endl;

    for (int i = 0; i < N; i++) {
        // display adjacent vertices of vertex i
        display_AdjList(diagraph.head[i], i);
    }

    return 0;
}
```

Output

```
Graph adjacency list
(start_vertex, end_vertex, weight):
(0, 2, 4) (0, 1, 2)
(1, 4, 3)
(2, 3, 2)
(3, 1, 4)
(4, 3, 3)
```

ILO B.1

Code

```cpp
#include <string>
#include <vector>
#include <iostream>
#include <set>
#include <map>
#include <stack>

template <typename T>
class Graph;

// Step 2: Edge struct
template <typename T>
struct Edge
{
    size_t src;
    size_t dest;
    T weight;

    // To compare edges, only compare their weights,
    // and not the source/destination vertices
    inline bool operator<(const Edge<T> &e) const
    {
        return this->weight < e.weight;
    }
    inline bool operator>(const Edge<T> &e) const
    {
        return this->weight > e.weight;
    }
};

// Step 3: Overload << operator
template <typename T>
std::ostream &operator<<(std::ostream &os, const Graph<T> &G)
{
    for (auto i = 1; i < G.vertices(); i++)
    {
        os << i << ": \t";
        auto edges = G.outgoing_edges(i);
        for (auto &e:Edge<T>& : edges)
            os << "{" << e.dest << ": " << e.weight << "}, ";
        os << std::endl;
    }
```

```
    return os;
}

// Step 4: Graph class implementation using edge list
template <typename T>
class Graph
{
public:
    // Initialize the graph with N vertices
    Graph(size_t N) : V(N)
    {
    }

    // Return number of vertices in the graph
    auto vertices() const ->size_t
    {
        return V;
    }

    // Return all edges in the graph
    auto &edges() const ->const vector<Edge<T>> &
    {
        return edge_list;
    }

    void add_edge(Edge<T> &&e)
    {
        // Check if the source and destination vertices are within range
        if (e.src >= 1 && e.src <= V && e.dest >= 1 && e.dest <= V)
            edge_list.emplace_back(e);
        else
            std::cerr << "Vertex out of bounds" << std::endl;
    }

    // Returns all outgoing edges from vertex v
    auto outgoing_edges(size_t v) const ->vector<Edge<T>>
    {
        std::vector<Edge<T>> edges_from_v;
        for (auto &e :const Edge<T> & : edge_list)
```

```
        {
            if (e.src == v)
            {
                edges_from_v.emplace_back(e);
            }
        }
        return edges_from_v;
    }

    // Overloads the << operator so a graph can be written directly to a stream
    template <typename U>
    friend std::ostream &operator<<(std::ostream &os, const Graph<U> &G);

private:
    size_t V; // Stores number of vertices in graph
    std::vector<Edge<T>> edge_list;
};

// Step 5: Depth-First Search function
template <typename T>
auto depth_first_search(const Graph<T> &G, size_t dest) ->vector<unsigned long long>
{
    std::stack<size_t> stack;
    std::vector<size_t> visit_order;
    std::set<size_t> visited;

    stack.push(1); // Assume that DFS always starts from vertex ID 1

    while (!stack.empty())
    {
        auto current_vertex :size_t = stack.top();
        stack.pop();

        // If the current vertex hasn't been visited in the past
        if (visited.find(current_vertex) == visited.end())
        {
```

```
118             visited.insert(current_vertex);
119             visit_order.push_back(current_vertex);
120
121             for (auto e :Edge<T> : G.outgoing_edges(current_vertex))
122             {
123                 // If the vertex hasn't been visited, insert it in the stack.
124                 if (visited.find(e.dest) == visited.end())
125                 {
126                     stack.push(e.dest);
127                 }
128             }
129         }
130     }
131     return visit_order;
132 }
133
134 // Step 6: Create reference graph
135 template <typename T>
136 auto create_reference_graph() ->Graph<T>
137 {
138     Graph<T> G(9);
139     std::map<unsigned, std::vector<std::pair<size_t, T>>> edges;
140
141     edges[1] = {{2, 0}, {5, 0}};
142     edges[2] = {{1, 0}, {5, 0}, {4, 0}};
143     edges[3] = {{4, 0}, {7, 0}};
144     edges[4] = {{2, 0}, {3, 0}, {5, 0}, {6, 0}, {8, 0}};
145     edges[5] = {{1, 0}, {2, 0}, {4, 0}, {8, 0}};
146     edges[6] = {{4, 0}, {7, 0}, {8, 0}};
147     edges[7] = {{3, 0}, {6, 0}};
148     edges[8] = {{4, 0}, {5, 0}, {6, 0}};
149
150     for (auto &i :pair<...> & : edges)
151         for (auto &j :pair<unsigned long long, T> & : i.second)
152             G.add_edge(Edge<T>{i.first, j.first, j.second});
153
```

```
153
154         return G;
155     }
156
157     // Step 7: Test function and driver code
158     template <typename T>
159     void test_DFS()
160     {
161         // Create an instance of and print the graph
162         auto G :Graph<unsigned> = create_reference_graph<unsigned>();
163         std::cout << G << std::endl;
164
165         // Run DFS starting from vertex ID 1 and print the order
166         // in which vertices are visited.
167         std::cout << "DFS Order of vertices: " << std::endl;
168         auto dfs_visit_order :vector<size_t> = depth_first_search(G, dest: 1);
169         for (auto v :size_t : dfs_visit_order)
170             std::cout << v << std::endl;
171     }
172
173     int main()
174     {
175         using T = unsigned;
176         test_DFS<T>();
177         return 0;
178     }
```

Output

```
1:      {2: 0}, {5: 0},
2:      {1: 0}, {5: 0}, {4: 0},
3:      {4: 0}, {7: 0},
4:      {2: 0}, {3: 0}, {5: 0}, {6: 0}, {8: 0},
5:      {1: 0}, {2: 0}, {4: 0}, {8: 0},
6:      {4: 0}, {7: 0}, {8: 0},
7:      {3: 0}, {6: 0},
8:      {4: 0}, {5: 0}, {6: 0},

DFS Order of vertices:
1
5
8
6
7
3
4
2

Process finished with exit code 0
```

ILO B.2

Code

```cpp
#include <string>
#include <vector>
#include <iostream>
#include <set>
#include <map>
#include <queue>

template <typename T>
class Graph;

// Step 2
template <typename T>
struct Edge
{
    size_t src;
    size_t dest;
    T weight;

    inline bool operator<(const Edge<T> &e) const
    {
        return this->weight < e.weight;
    }
    inline bool operator>(const Edge<T> &e) const
    {
        return this->weight > e.weight;
    }
};

// Step 3
template <typename T>
std::ostream &operator<<(std::ostream &os, const Graph<T> &G)
{
    for (auto i = 1; i < G.vertices(); i++)
    {
        os << i << ": \t";
        auto edges = G.outgoing_edges(i);
        for (auto &e:Edge<T>& : edges)
            os << "{" << e.dest << ": " << e.weight << "}, ";
        os << std::endl;
    }
    return os;
}
```

```cpp

// Step 4
template <typename T>
class Graph
{
public:
    Graph(size_t N) : V(N) {}

    auto vertices() const ->size_t
    {
        return V;
    }

    auto &edges() const ->const vector<Edge<T>> &
    {
        return edge_list;
    }

    void add_edge(Edge<T> &&e)
    {
        if (e.src >= 1 && e.src <= V && e.dest >= 1 && e.dest <= V)
            edge_list.emplace_back(e);
        else
            std::cerr << "Vertex out of bounds" << std::endl;
    }

    auto outgoing_edges(size_t v) const ->vector<Edge<T>>
    {
        std::vector<Edge<T>> edges_from_v;
        for (auto &e :const Edge<T> & : edge_list)
        {
            if (e.src == v)
            {
                edges_from_v.emplace_back(e);
            }
        }
        return edges_from_v;
    }

```

```
81
82      template <typename U>
83      friend std::ostream &operator<<(std::ostream &os, const Graph<U> &G);
84
85  private:
86      size_t V;
87      std::vector<Edge<T>> edge_list;
88  };
89
90  // Step 5
91  template <typename T>
92  auto create_reference_graph() ->Graph<T>
93  {
94      Graph<T> G(9);
95      std::map<unsigned, std::vector<std::pair<size_t, T>>> edges;
96
97      edges[1] = {{2, 2}, {5, 3}};
98      edges[2] = {{1, 2}, {5, 5}, {4, 1}};
99      edges[3] = {{4, 2}, {7, 3}};
100     edges[4] = {{2, 1}, {3, 2}, {5, 2}, {6, 4}, {8, 5}};
101     edges[5] = {{1, 3}, {2, 5}, {4, 2}, {8, 3}};
102     edges[6] = {{4, 4}, {7, 4}, {8, 1}};
103     edges[7] = {{3, 3}, {6, 4}};
104     edges[8] = {{4, 5}, {5, 3}, {6, 1}};
105
106     for (auto &i :pair<...> & : edges)
107         for (auto &j :pair<unsigned long long, T> & : i.second)
108             G.add_edge(Edge<T>{i.first, j.first, j.second});
109
110     return G;
111 }
112
113 // Step 6
114 template <typename T>
115 auto breadth_first_search(const Graph<T> &G, size_t dest) ->vector<unsigned long long>
116 {
117     std::queue<size_t> queue;
```

```
    std::queue<size_t> queue;
    std::vector<size_t> visit_order;
    std::set<size_t> visited;

    queue.push( x: 1); // Assume that BFS always starts from vertex ID 1

    while (!queue.empty())
    {
        auto current_vertex :size_t  = queue.front();
        queue.pop();

        // If the current vertex hasn't been visited in the past
        if (visited.find(current_vertex) == visited.end())
        {
            visited.insert(current_vertex);
            visit_order.push_back(current_vertex);

            for (auto e :Edge<T>  : G.outgoing_edges(current_vertex))
                queue.push(e.dest);
        }
    }
    return visit_order;
}

// Step 7
template <typename T>
void test_BFS()
{
    // Create an instance of and print the graph
    auto G :Graph<unsigned>  = create_reference_graph<unsigned>();
    std::cout << G << std::endl;

    // Run BFS starting from vertex ID 1 and print the order
    // in which vertices are visited.
    std::cout << "BFS Order of vertices: " << std::endl;
    auto bfs_visit_order :vector<size_t>  = breadth_first_search(G,  dest: 1);
    for (auto v :size_t  : bfs_visit_order)
        std::cout << v << std::endl;
}
```

```
155     }
156
157     int main()
158     {
159         using T = unsigned;
160         test_BFS<T>();
161         return 0;
162     }
```

Output

```
1:      {2: 2}, {5: 3},
2:      {1: 2}, {5: 5}, {4: 1},
3:      {4: 2}, {7: 3},
4:      {2: 1}, {3: 2}, {5: 2}, {6: 4}, {8: 5},
5:      {1: 3}, {2: 5}, {4: 2}, {8: 3},
6:      {4: 4}, {7: 4}, {8: 1},
7:      {3: 3}, {6: 4},
8:      {4: 5}, {5: 3}, {6: 1},

BFS Order of vertices:
1
2
5
4
8
3
6
7
```

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

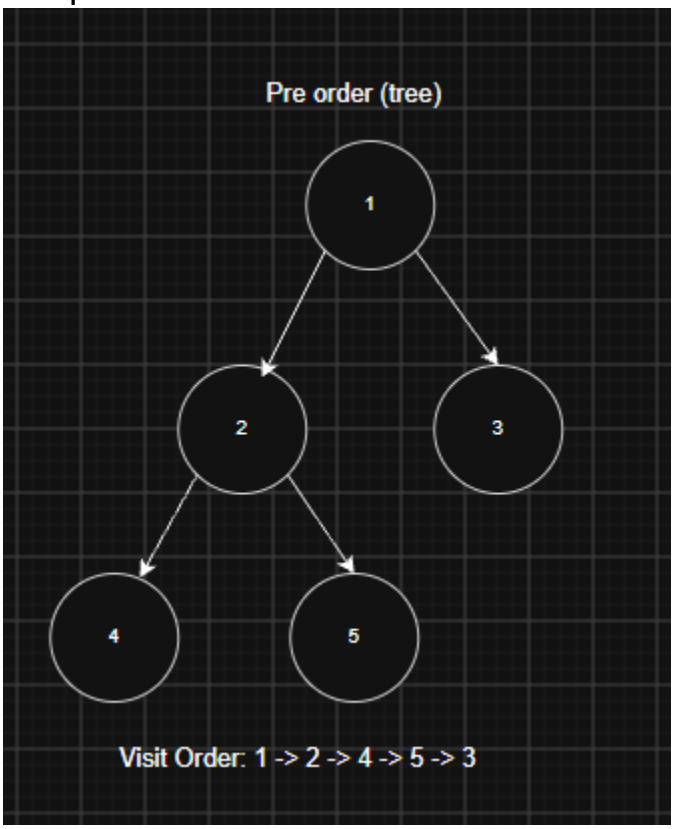
ILO C

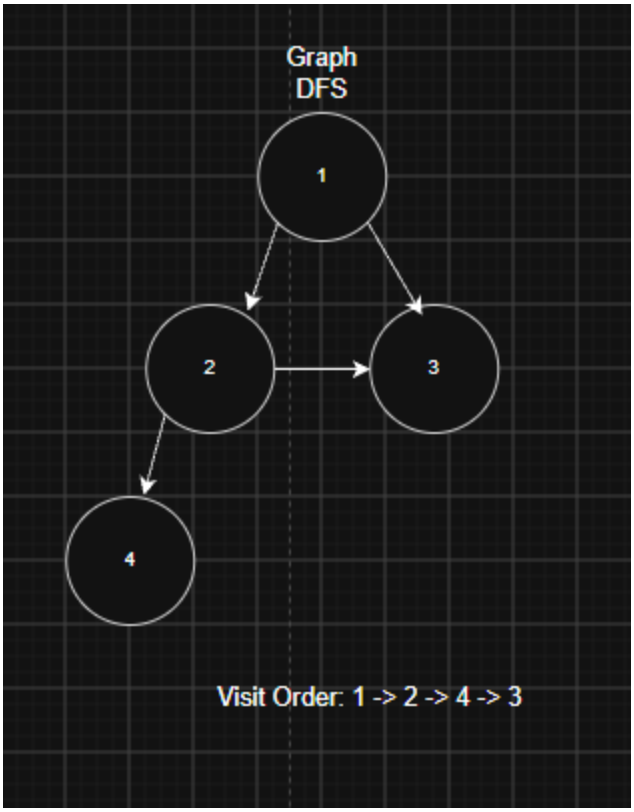
1. A person wants to visit different locations indicated on a map. He starts from one location (vertex) and wants to visit every vertex until it finishes from one vertex, backtracks, and then explore other vertex from the same vertex. Discuss which algorithm would be most helpful to accomplish this task.

- **It would be the Depth First Search algorithm because it explores as deeply as possible along a**

single path until reaching a dead end before backtracking to explore adjacent unvisited branches.

2. Identify the equivalent of DFS in traversal strategies for trees. To efficiently answer this question, provide a graphical comparison, examine pseudocode and code implementation.

- **The equivalent would be the Pre-Order Traversal.**
- 

- 

- **Pseudocode:**
- 1. Visit Current Node
- 2. Go deep down the left path
- 3. Backtrack and go down the right path.
- **Code:**

-



```cpp
#include <iostream>

struct Node {
    int data;
    Node* left;
    Node* right;

    Node(int val) {
        data = val;
        left = nullptr;
        right = nullptr;
    }
};

void treeDFS(Node* root) {
    if (root == nullptr) return;

    std::cout << root->data << " "; // Visit root
    treeDFS(root->left);           // Explore left child
    treeDFS(root->right);          // Explore right child
}
```

3. In the performed code, what data structure is used to implement the Breadth First Search?
   - **A queue is used to implement the Breadth-First Search.**

4. How many times can a node be visited in the BFS?
   - **A node is visited exactly once.**

Conclusion:
In this activity I learned how to create a C++ code for graph implementation utilizing matrix and adjacency list. I also implemented graph traversal algorithms such as breadth-first and depth-first search. After that, I also demonstrated an understanding of graph implementation, operations, and traversal methods. With that, I also need to keep learning about graphs to deepen my understanding so that I am able to know and be familiar with it.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Gemini**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Concept Explanation**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **I used ai to explain the concepts to me more simply.**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.